parser_demo

Step 1: Open Python

You can use:

- **IDLE** (comes with Python)
- **VS Code**
- **Jupyter Notebook**
- **Google Colab**
- Or simply the terminal/command prompt

Step 2: Create a Python file

Create a file named parser_demo.py and paste:

```
import ast

# An artificial language: one fixed parse
tree = ast.parse("x = a + b * c")
print(ast.dump(tree, indent=2))

# A natural-language sentence: just a string
sent = "I saw the man with the telescope"
print(sent.split())
```

Step 3: Run it

In the terminal:

```
python parser_demo.py
```

Expected Output

The AST (Abstract Syntax Tree) will show something like:

```
Assign(
    targets=[
        Name(id='x', ctx=Store())],
    value=BinOp(
        left=Name(id='a', ctx=Load()),
        op=Add(),
        right=BinOp(
            left=Name(id='b', ctx=Load()),
            op=Mult(),
             right=Name(id='c', ctx=Load()))))
```

Notice:

```
a + (b * c)
```

Python automatically groups b * c together because **multiplication has higher precedence than addition**.

Exercise Answer

The tree places:

```
b * c
```

as the **right child** of the addition.

Current parse:

```
x = a + (b * c)
```

If you want it parsed the other way:

```
x = (a + b) * c
```

use parentheses:

```
tree = ast.parse("x = (a + b) * c")
print(ast.dump(tree, indent=2))
```

Now the AST will first group a + b, and then multiply the result by c.

Quick Classroom Demonstration

```
print(a + b * c)      # interpreted as a + (b*c)
print((a + b) * c)    # parentheses change the parse
```

Example:

```
a = 2
b = 3
c = 4

print(a + b * c)      # 14
print((a + b) * c)    # 20
```

This clearly shows how the parser builds different trees based on operator precedence and parentheses.